

CSE4067

Introduction to Blockchain Programming

Goshgar Can Ismayilov

goshgar.ismayilov@boun.edu.tr

<https://www.linkedin.com/in/gismayilov/>

26.11.2024

DoS: Unbounded Operations

```
// INSECURE
contract NaiveBank {
    struct Account {
        address addr;
        uint balance;
    }

    Account accounts[];
    function applyInterest() returns (uint) {
        for (uint i = 0; i < accounts.length; i++) {
            // apply 5 percent interest
            accounts[i].balance = accounts[i].balance * 105 / 100;
        }
        return accounts.length;
    }

    function openAccount() public returns (uint) { ... }
}
```

DoS: Unbounded Operations

```
uint256 winnersLength = winners.length; // cache length
for (uint256 i; i < winnersLength;) {
    uint256 amount = totalAmount * percentages[i] / BASIS_POINTS;
    erc20.safeTransfer(winners[i], amount);
    unchecked {
        ++i;
    }
}
```

In more extreme cases, if there are “too many winners” the gas cost of running the distribution function and iterating through 2 huge arrays twice might be bigger than the block gas limit, in that case, rewards can’t be distributed and the distribution function will be broken because it will always be reverted due to out of gas error.

DoS: Griefing

```
// INSECURE
```

```
for (uint i = 0; i < investors.length; i++) {  
    if (investors[i].invested >= min_investment) {
```

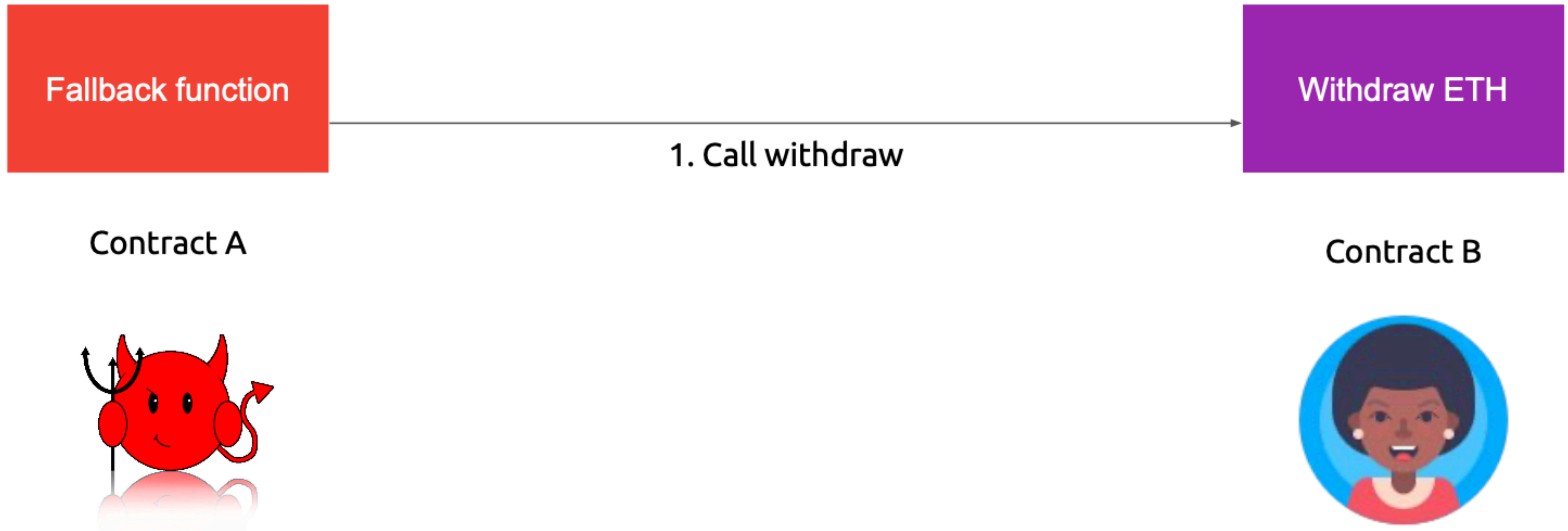
```
        if (!(investors[i].addr.send(investors[i].dividendAmount))) {  
            revert();  
        }
```

```
        investors[i] = newInvestor;
```

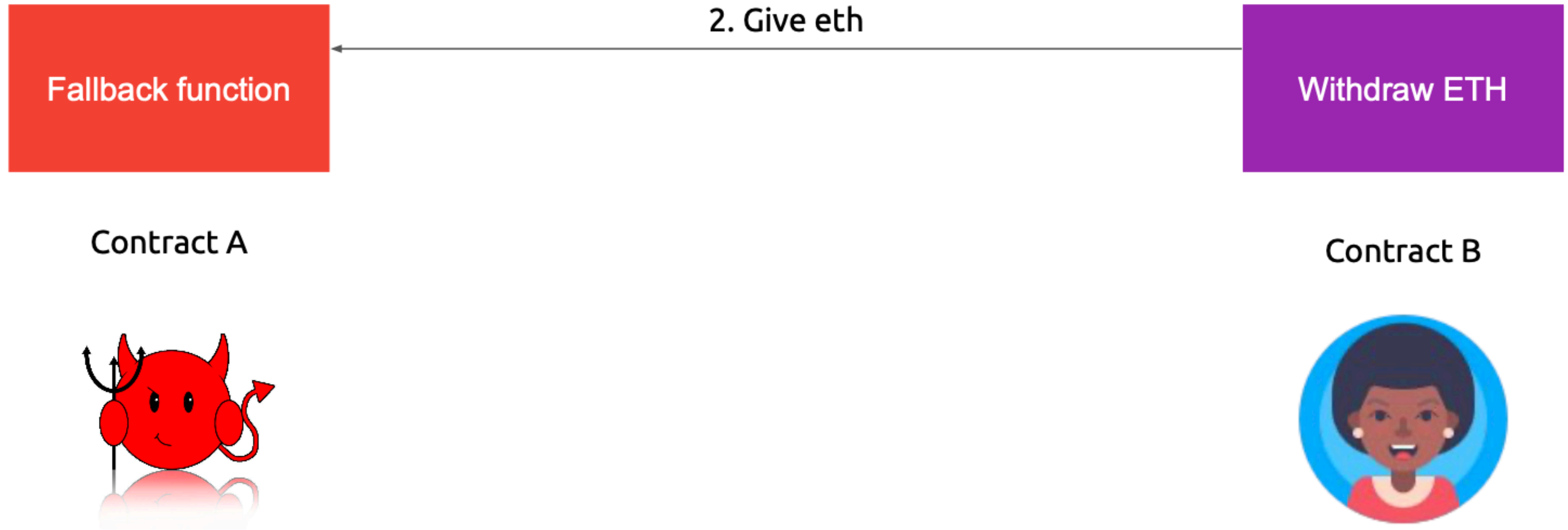
```
    }
```

```
}
```

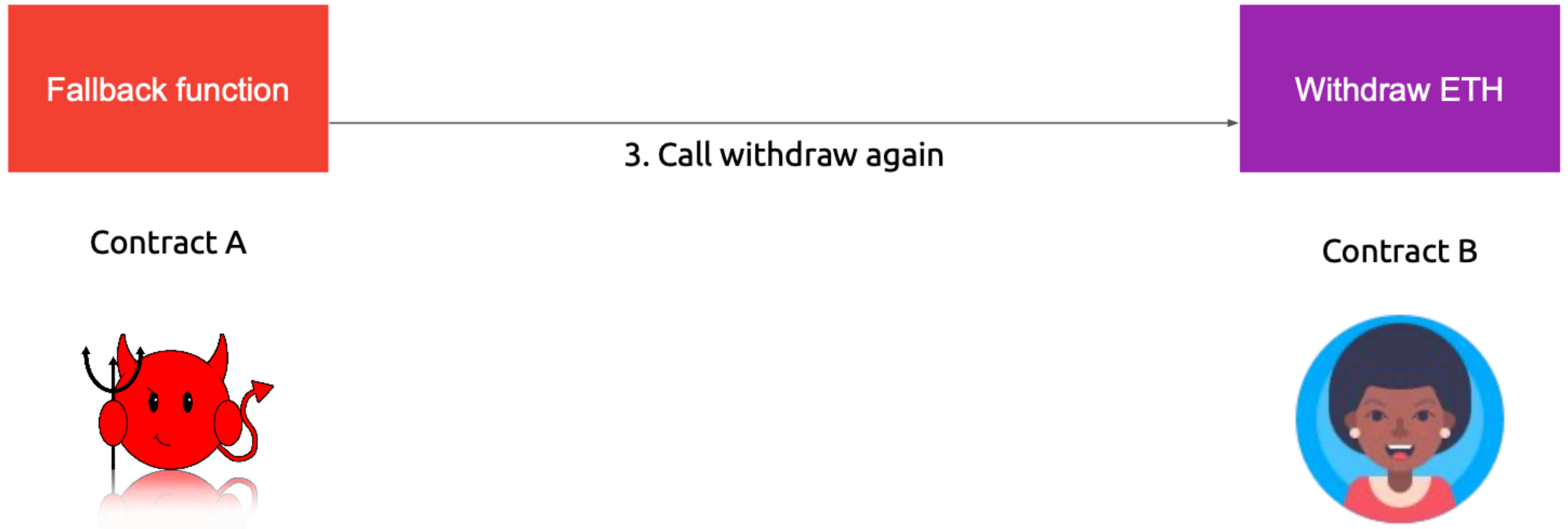
Re-Entrancy Attack



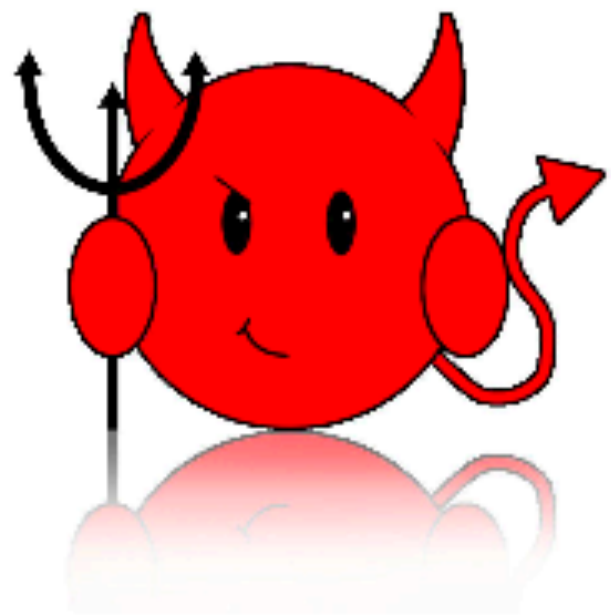
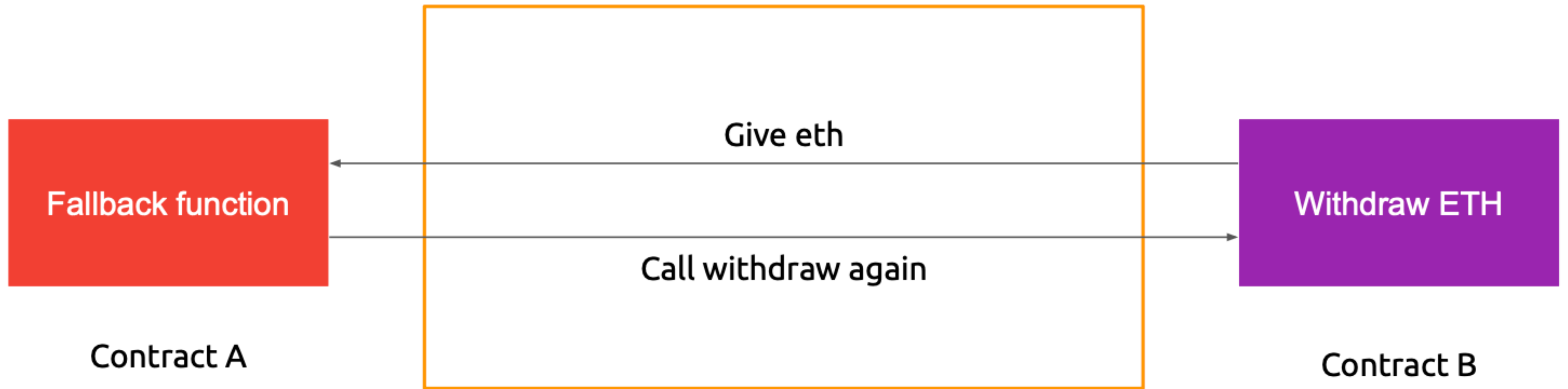
Re-Entrancy Attack



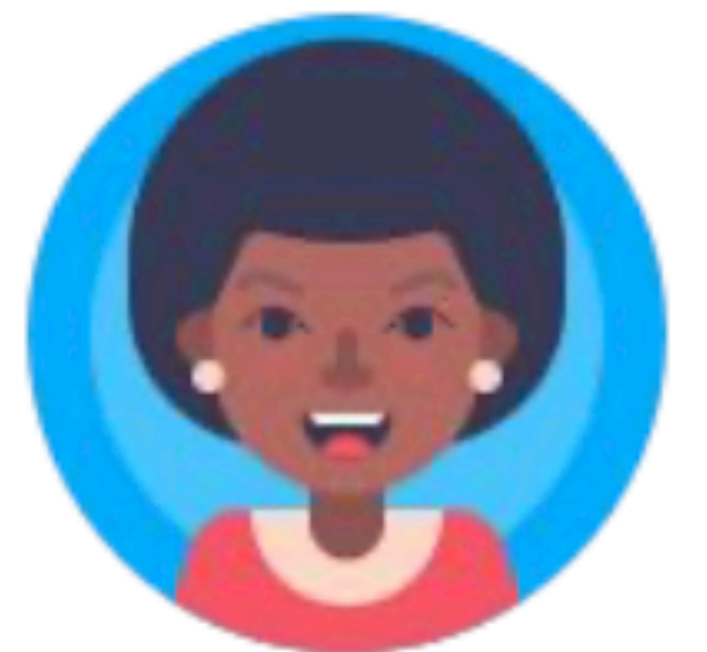
Re-Entrancy Attack



Re-Entrancy Attack



Loop of function calls



Re-Entrancy Attack

```
// INSECURE

mapping (address => uint) private userBalances;

function withdrawBalance() public {

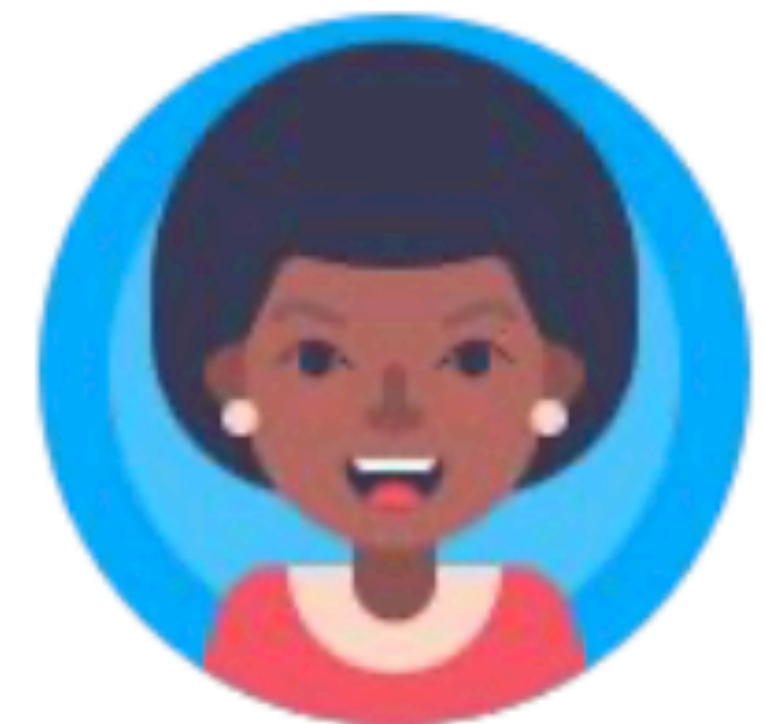
    uint amountToWithdraw = userBalances[msg.sender];

    require(msg.sender.call.value(amountToWithdraw)());

    userBalances[msg.sender] = 0;

}
```

Contract B



Re-Entrancy Attack

Begin attack by sending msg.value eth

```
// INSECURE
```

```
mapping (address => uint) private userBalances;
```

```
function withdrawBalance() public {
```

```
    uint amountToWithdraw = userBalances[msg.sender];
```

```
    require(msg.sender.call.value(amountToWithdraw)());
```

```
    userBalances[msg.sender] = 0;
```

```
}
```

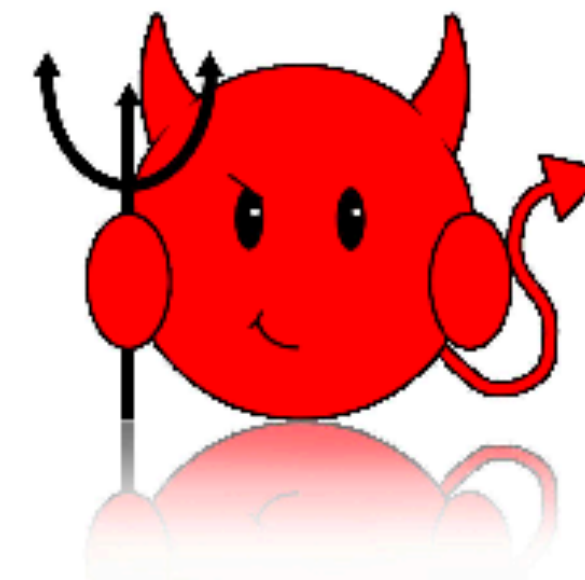
```
function () payable {
```

```
    if (victim.balance >= msg.value) {
```

```
        victim.withdrawBalance();
```

```
    }
```

```
}
```



Re-Entrancy Attack

The DAO (distributed autonomous organization)

- Designed by slock.it in 2016
- Purpose: Create a population of stakeholders
- Stake (in the form of DAO tokens) enables them to participate in decision making
- Decision-making to choose which proposals to fund

The DAO

The DAO's Mission: To blaze a new path in business organization for the betterment of its members, existing simultaneously nowhere and everywhere and operating solely with the steadfast iron will of **unstoppable code.**

Re-Entrancy Attack

- June 12: The reentrancy bug is identified (but stakeholders are reassured)
- June 17: Attacker exploits it draining ~\$50Million at the time of the attack
- July 15: Ethereum Classic manifesto
- July 19: “Hard Fork” neutralizes attacker’s smart contract

I think TheDAO is getting drained right now

self.ethereum

Submitted 1 year ago by ledgerwatch

Re-Entrancy Attack

```
// SECURE
```

```
mapping (address => uint) private userBalances;
```

```
function withdrawBalance() public {
```

```
    uint amountToWithdraw = userBalances[msg.sender];
```

```
    userBalances[msg.sender] = 0;
```

```
    msg.transfer(amountToWithdraw);
```

```
}
```

- Finish all internal work (ie. state changes) and then call external functions
- Checks-Effects-Interactions Pattern
 - Mutexes
 - Pull-push pattern
- Use **transfer** or **send** instead of **call**

Checks-Effects-Interactions Patterns

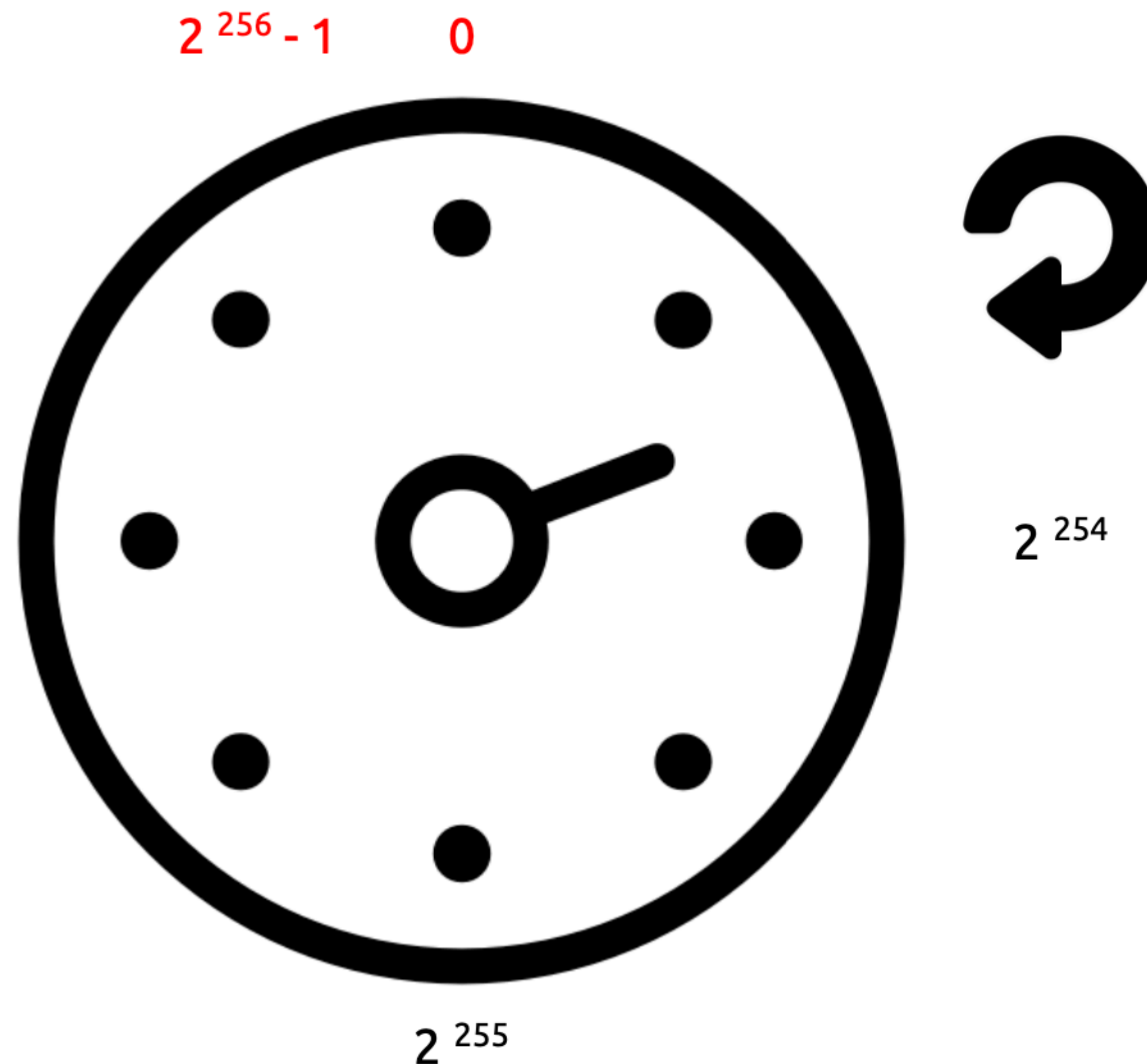
The CHECKS-EFFECTS-INTERACTIONS programming pattern, commonly abbreviated as CEI, offers a structured way of writing contract functions to avoid potential pitfalls. In essence, this pattern suggests the following order of operations inside a function:

1. **Checks:** Validate all conditions and prerequisites. For example, check if the caller has the necessary permissions or if there's enough ether sent.
2. **Effects:** Update the state variables.
3. **Interactions:** Interact with other contracts, send Ether, or conduct external function calls.

Integer Overflow and Underflow



Integer Overflow and Underflow



Integer Overflow and Underflow

```
// INSECURE
```

```
function withdraw(uint256 _value) {  
    require(balanceOf[msg.sender] >= _value);  
    msg.sender.call.value(_value)();  
    balanceOf[msg.sender] -= _value;  
}
```

Integer Overflow and Underflow

// INSECURE

```
function withdraw(uint256 _value) {  
    require(balanceOf[msg.sender] >= _value);  
    msg.sender.call.value(_value)();  
    balanceOf[msg.sender] -= _value;  
}
```

```
function attack() {  
    victim.donate.value(1)();  
    victim.withdraw(1);  
}  
  
function() {  
    if (performAttack) {  
        performAttack = false;  
        victim.withdraw(1);  
    }  
}
```

Integer Overflow and Underflow

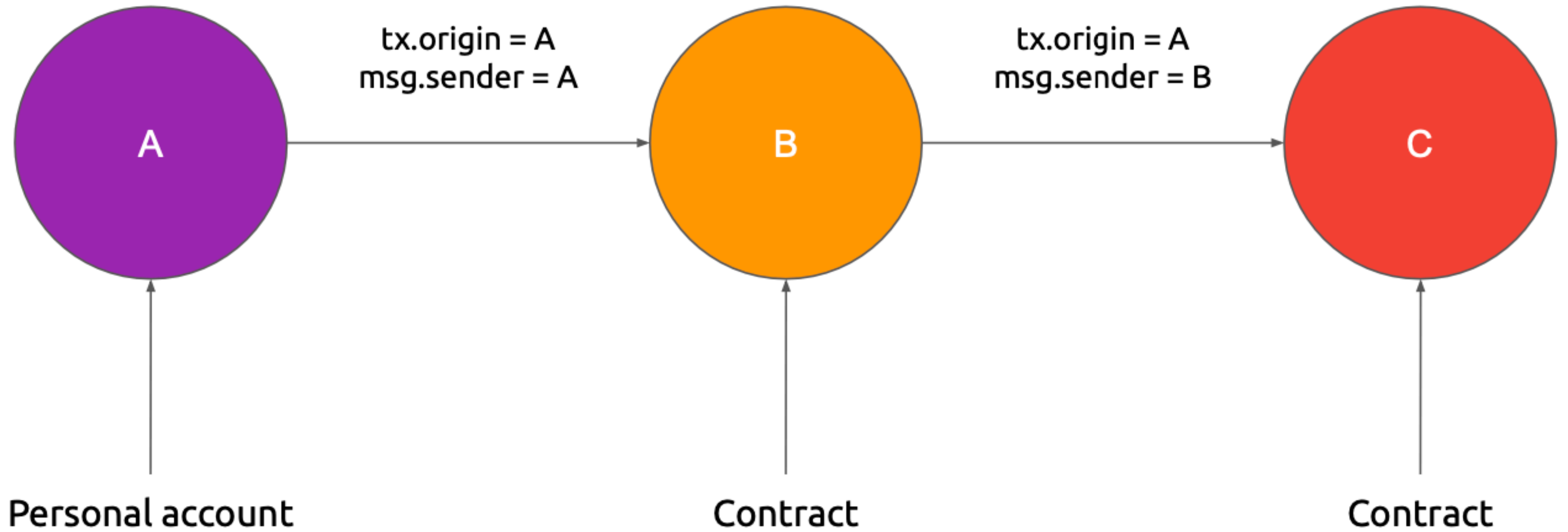
Use OpenZeppelin's SafeMath library

```
// OpenZeppelin: SafeMath.sol
```

```
function add(uint256 a, uint256 b) internal pure returns  
(uint256) {  
    uint256 c = a + b;  
    require(c >= a, "SafeMath: addition overflow");  
  
    return c;  
}
```

```
function sub(uint256 a, uint256 b) internal pure returns  
(uint256) {  
    require(b <= a, "SafeMath: subtraction overflow");  
    uint256 c = a - b;  
  
    return c;  
}
```

Tx.Origin



Tx.Origin

```
// INSECURE
contract Bank {

    address owner;

    constructor() public {
        owner = msg.sender;
    }

    function sendTo(address receiver, uint amount) public {
        require(tx.origin == owner);
        receiver.call.value(amount)();
    }

}
```

Tx.Origin

```
// INSECURE
contract Bank {
    address owner;

    constructor() public {
        owner = msg.sender;
    }

    function sendTo(address payable receiver, uint amount)
    public {
        require(tx.origin == owner);
        receiver.call.value(amount)();
    }
}
```

```
function() external payable {
    victim.sendTo(attacker, msg.sender.balance);
}
```

A diagram illustrating a call from the Bank contract to an external payable function. A vertical line separates the two code blocks. An arrow points from the `require(tx.origin == owner);` line in the Bank contract to the `function() external payable {` block. Another arrow points from the `victim.sendTo(attacker, msg.sender.balance);` line in the external function back to the top of the Bank contract, indicating a return path.

Pull over Push

Do not transfer ether to users (push) but let the users withdraw (pull) their funds.

// BAD DESIGN (PUSH)

```
function bid() payable {
    require(msg.value >= highestBid);

    if (highestBidder != address(0)) {
        highestBidder.transfer(highestBid);
    }

    highestBidder = msg.sender;
    highestBid = msg.value;
}
```

// PULL DESIGN

```
function bid() payable external {
    require(msg.value >= highestBid);

    if (highestBidder != address(0)) {
        refunds[highestBidder] += highestBid;
    }

    highestBidder = msg.sender;
    highestBid = msg.value;
}

function withdrawRefund() external {
    uint refund = refunds[msg.sender];
    refunds[msg.sender] = 0;
    msg.sender.transfer(refund);
}
```

Fallback Functions

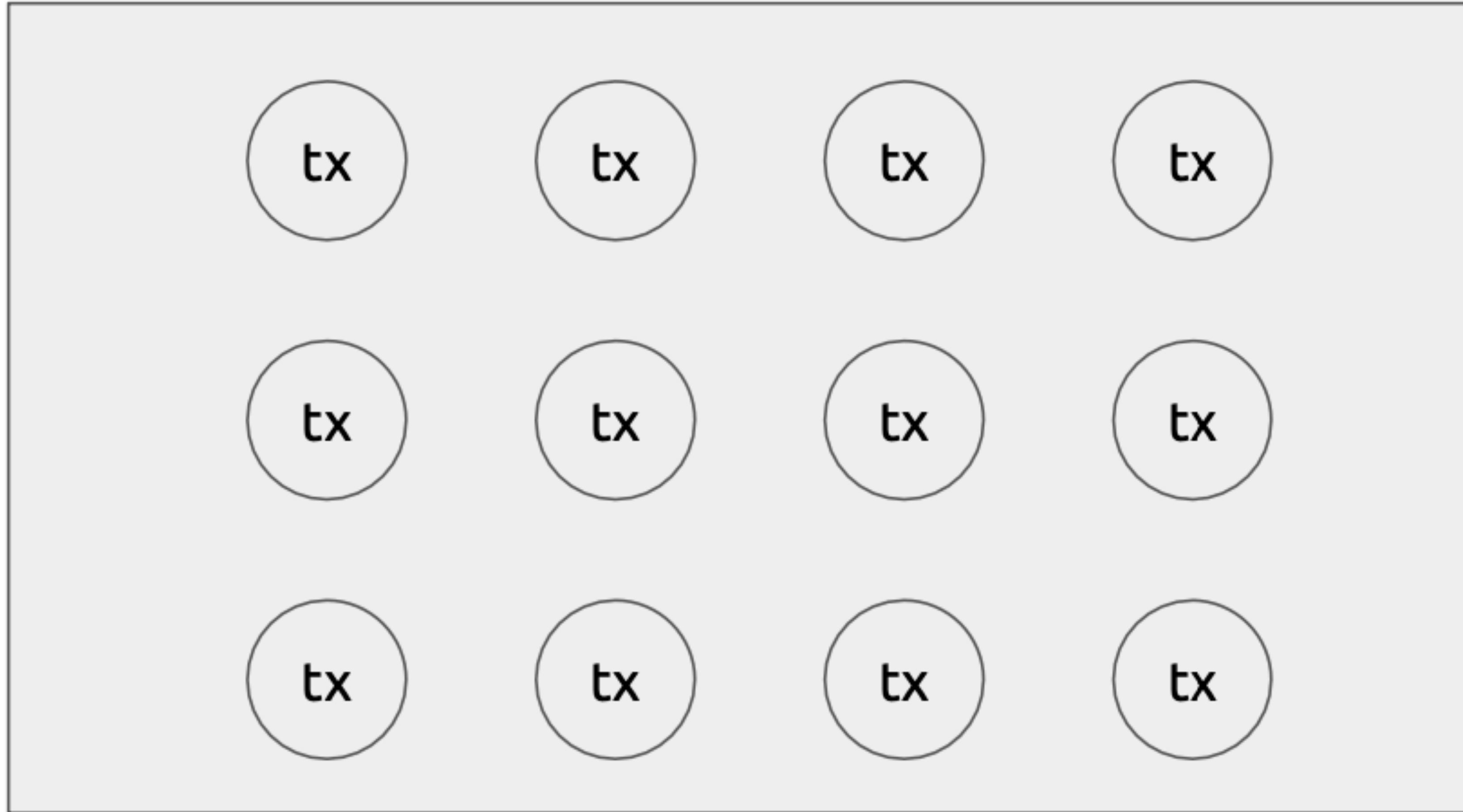
// BAD

```
function() payable {  
    balances[msg.sender] += msg.value;  
}
```

// GOOD

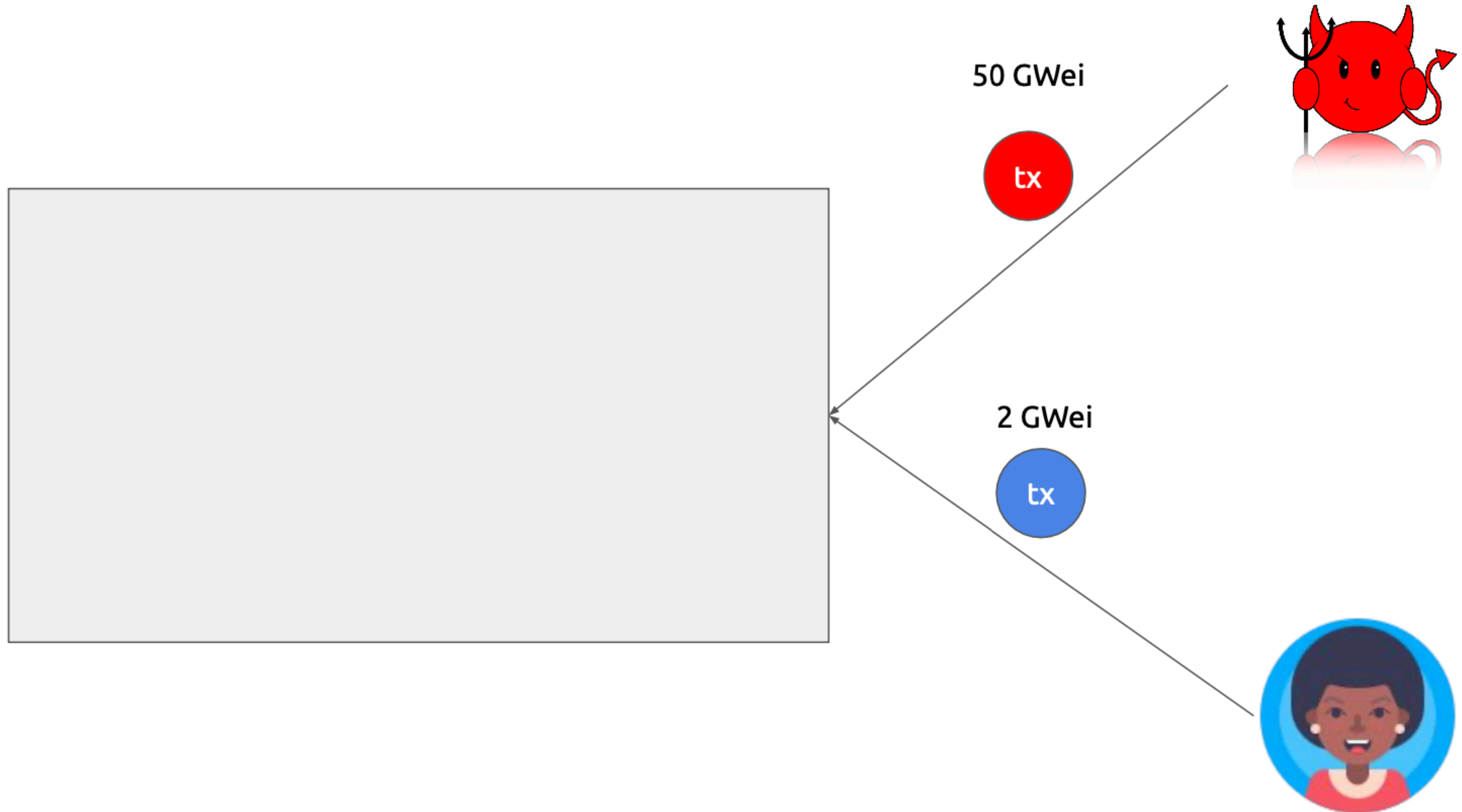
```
function deposit() payable external {  
    balances[msg.sender] += msg.value;  
}  
  
function() payable {  
    require(msg.data.length == 0);  
    emit LogDepositReceived(msg.sender);  
}
```

Front-Running

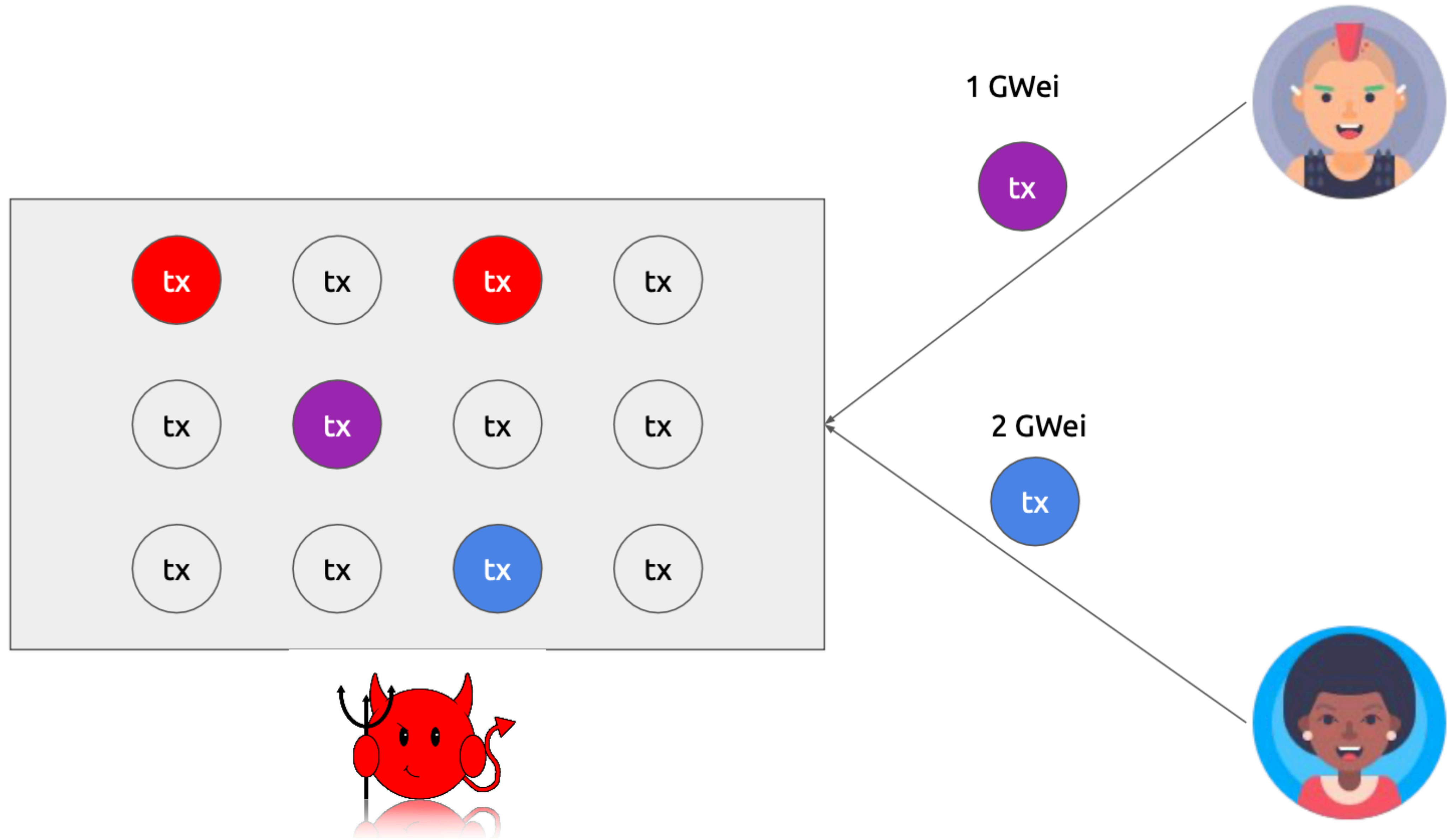


Miner: `sortByGasPrice(txs, 'desc')`

Front-Running



Front-Running



Front-Running

```
// INSECURE
```

```
function registerName(bytes32 name) public {  
    names[name] = msg.sender;  
}
```

Front-Running

```
// INSECURE
```

```
function commit(bytes32 commitment) public {  
    commitments[commitment] = msg.sender;  
}
```

```
function registerName(bytes32 name, bytes32 nonce) public {  
    require(commitments[makeCommitment(name, nonce)] == msg.sender, "Not found!");  
    names[name] = msg.sender;  
}
```

makeCommitment is a non-interactive cryptographic commitment function.

Properties:

- Binding. A commitment can be only opened to its committed value.
- Hiding. The commitment reveals no information about its committed value.

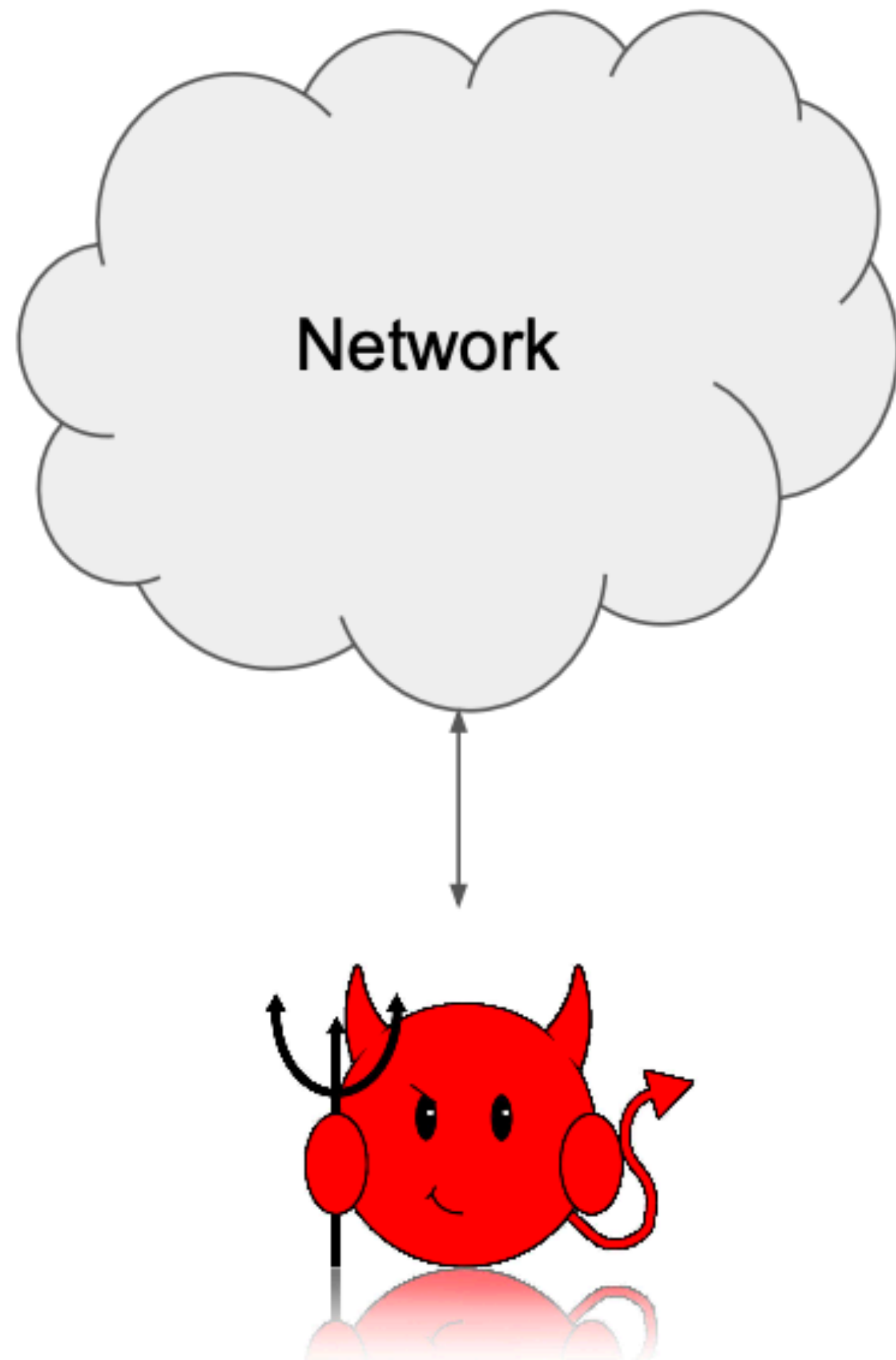
A way to implement is via a cryptographic hash function, e.g., keccak256

Note: nonce space should be large!

Front-Running



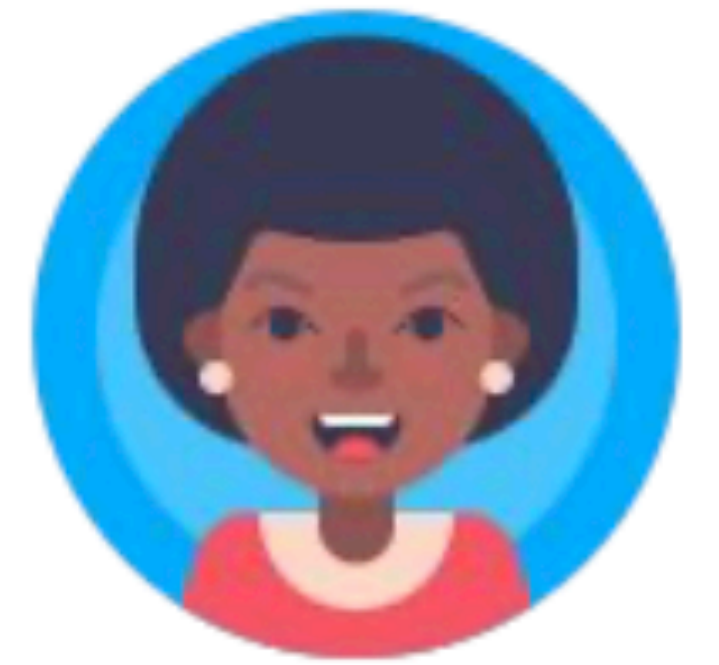
Front-Running



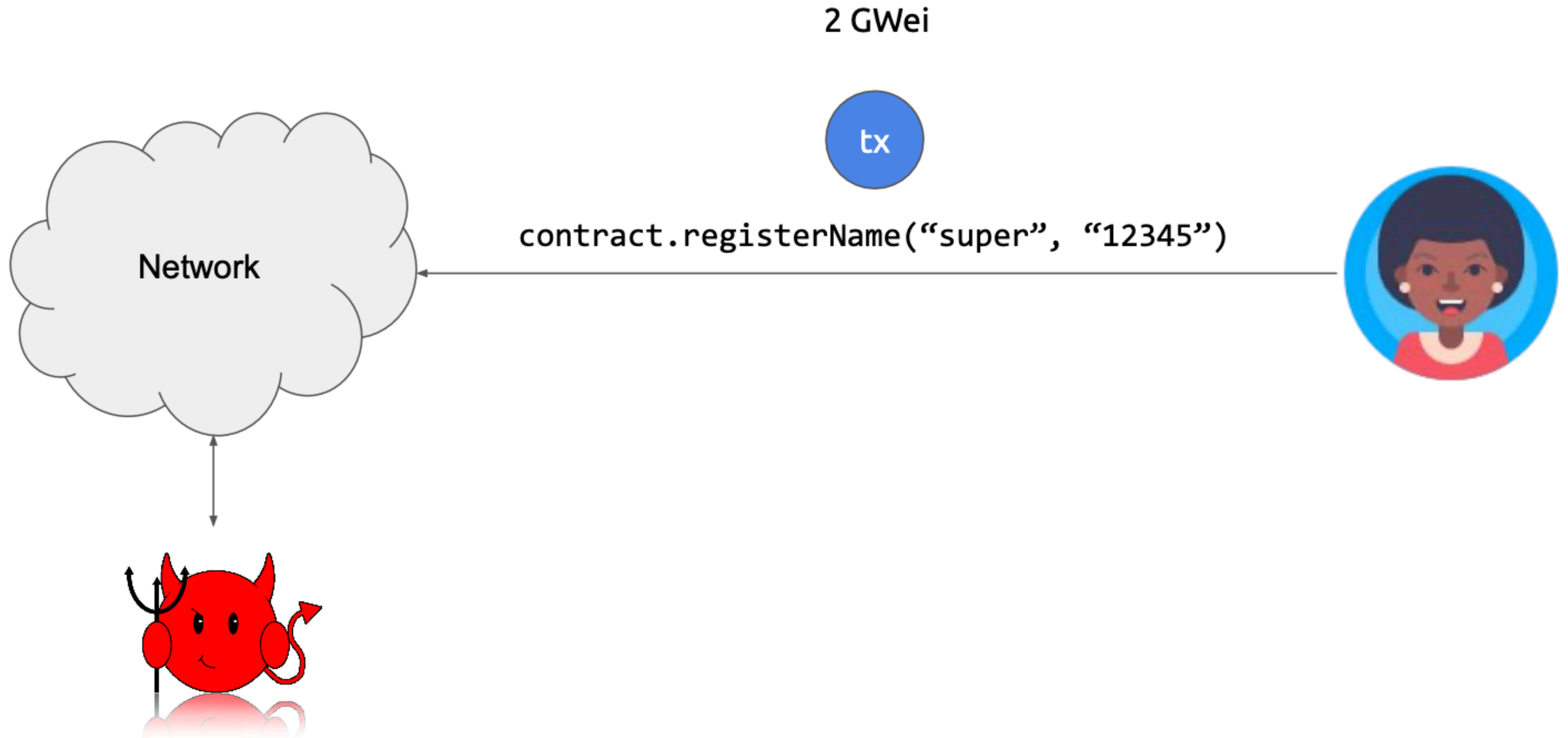
2 GWei



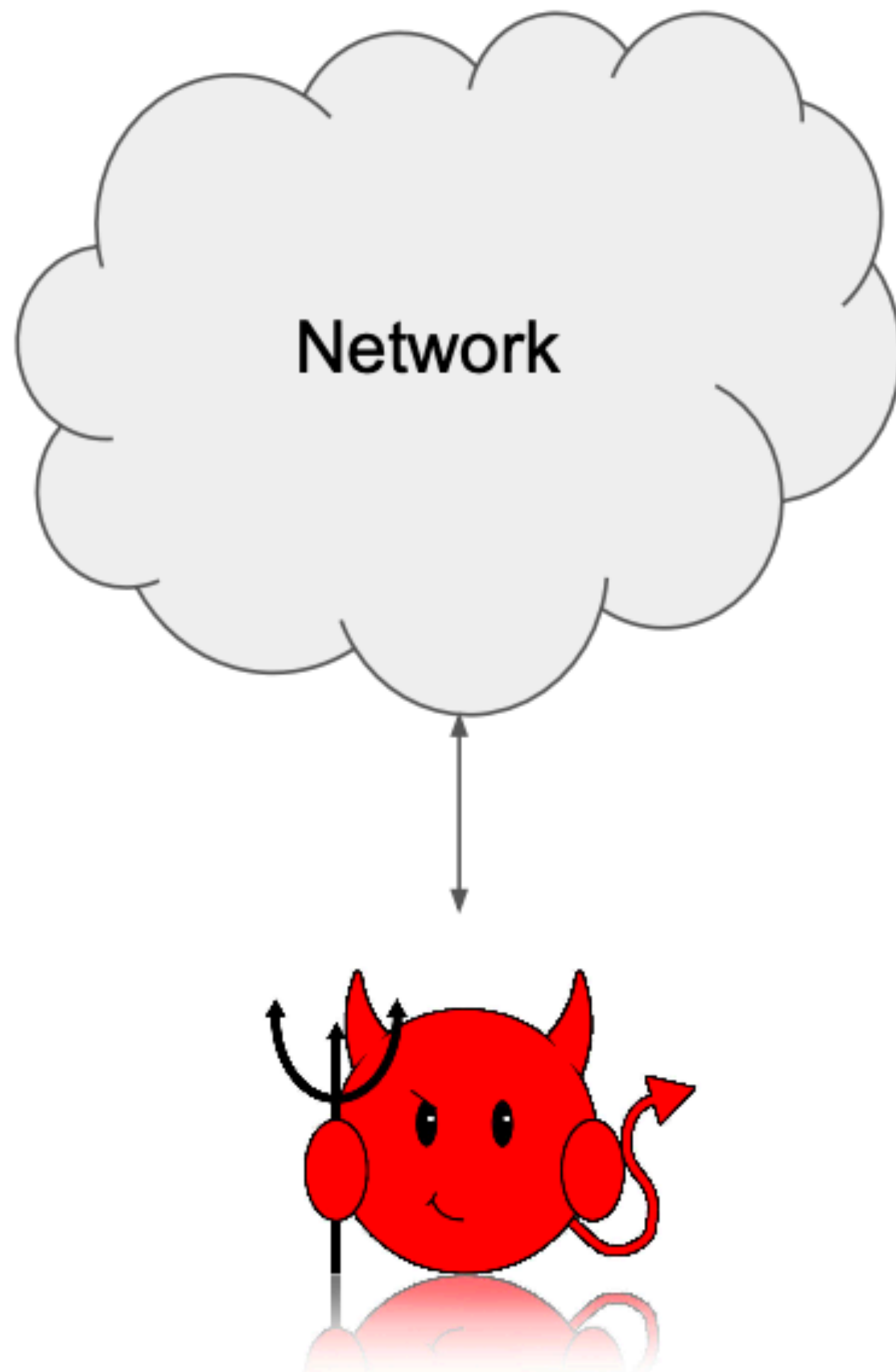
`contract.commit("9505cacb")`



Front-Running



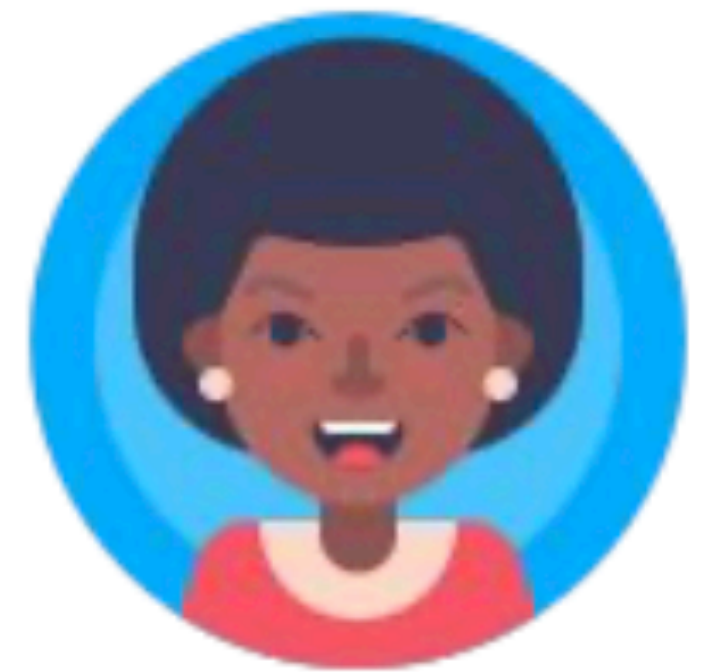
Front-Running



50 GWei



`contract.registerName("super", "12345")`



* sandwich attack

Randomness

- block.number
- block.timestamp
- block.hash
- block.difficulty
- block.coinbase
- block.gasLimit
- now
- msg.sender

They can be manipulated by a malicious miner.
They are shared within the same block to all users.

```
uint(keccak256(

|           |            |      |     |
|-----------|------------|------|-----|
| timestamp | msg.sender | hash | ... |
|-----------|------------|------|-----|

)) % n
```

Randomness

```
// INSECURE  
bool won = (block.number % 2) == 0;
```

```
// INSECURE  
uint random = uint(keccak256(block.timestamp)) % 2;
```

```
// INSECURE  
address seed1 = contestants[uint(block.coinbase) % totalTickets].addr;  
address seed2 = contestants[uint(msg.sender) % totalTickets].addr;  
uint seed3 = block.difficulty;  
bytes32 randHash = keccak256(seed1, seed2, seed3);  
uint winningNumber = uint(randHash) % totalTickets;  
address winningAddress = contestants[winningNumber].addr;
```

Randomness

Not that private :)

// INSECURE

`uint256 constant private` FACTOR =
1157920892373161954235709850086879078532699846656405640394575840079131296399;

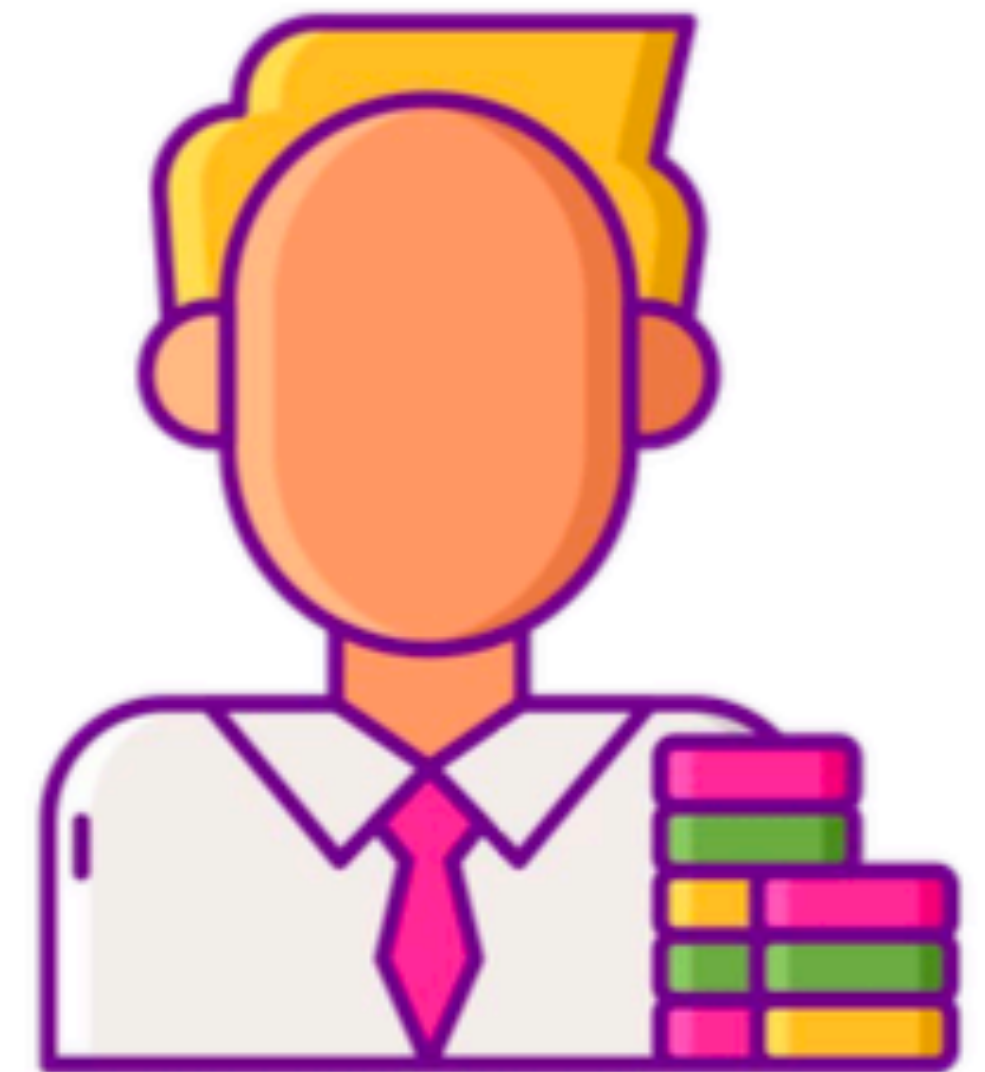
`function` rand(`uint` max) `constant private` returns (`uint256` result) {
 `uint256` factor = FACTOR * 100 / max;
 `uint256` lastBlockNumber = `block.number - 1`;
 `uint256` hashVal = `uint256`(`block.blockhash`(lastBlockNumber));
 return `uint256`((`uint256`(hashVal) / factor)) % max;
}

Randomness



Casino

1. Player makes a bet and the casino stores the `block.number` of the transaction



Player

Randomness



Casino

2. A few blocks later, player requests
from the casino to announce the
winning number



Player

Randomness

Validate block.number age!

3. Casino uses, as a source of randomness, the block.hash with a `block.number` produced after the bet is placed



Casino



Player

Safer Randomness

Commit - reveal schemes

Example:

- Casino and player commit each to a random value.
- Casino and player reveal their random values.
- Casino XORs the random values to a seed. The seed can be combined with the hash of a future block.

Gas Fairness

Crowdfunding Contract #1

R sets a threshold

Contract collects contributions

When balance exceeds threshold, it sends funds to R and returns any surplus to contributors.

Funding paid by last contributor

VS.

Crowdfunding Contract #2

R sets a threshold

Contract collects contributions

When balance exceeds threshold, it allows R to withdraw the threshold and return any surplus to contributors

Each contributor pays for withdrawal

Gas Fairness

```
contract RockPaperScissors {
    struct hand {
        address payable player;
        bytes32 c;
        uint val;
    }
    hand[] hands;

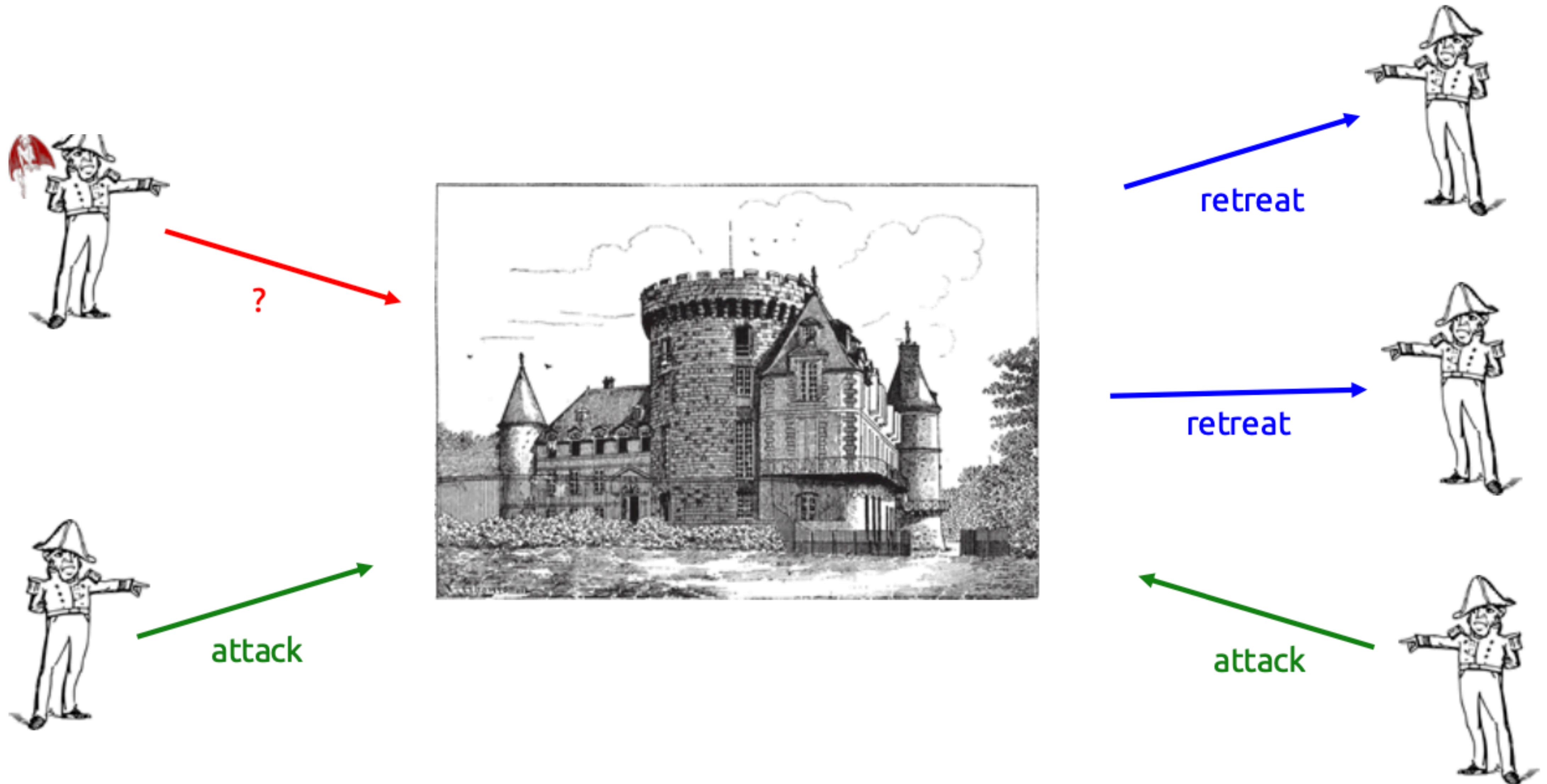
    function commit(uint value) payable public {
        require((value == 1 || value == 2 || value == 3) && (hands.length < 2));
        hands.push(hand(msg.sender, sha256(abi.encode(value)), 0));
    }

    function open(uint value) public {
        require(hands.length == 2);
        for (uint256 i = 0; i < 2; i++) {
            if (hands[i].c == sha256(abi.encode(value))) {
                hands[i].val = value;
            }
        }
        if (hands[0].val != 0 && hands[1].val != 0) {
            if (hands[0].val == hands[1].val) {
                hands[0].player.transfer(address(this).balance / 2);
                hands[1].player.transfer(address(this).balance / 2);
            }
            else {
                if ((hands[0].val == 1 && hands[1].val == 2) || (hands[0].val == 2 && hands[1].val == 3) || (hands[0].val == 3 && hands[1].val == 1)) {
                    hands[0].player.transfer(address(this).balance);
                }
                else {
                    hands[1].player.transfer(address(this).balance);
                }
            }
        }
        selfdestruct(msg.sender);
    }
}
```

Byzantine Generals Problem



Byzantine Generals Problem



Consensus in Distributed Systems

In simple terms, a blockchain is merely a replicated, distributed system where network participants try to **keep in sync** (i) **while accepting new transactions** (ii).

- i. In a distributed system, not every node has the same worldview at every moment. Hence, they need to **synchronize to achieve consistency**.
- ii. When a transaction enters a distributed system, it should **eventually be known by every participant**.

Definition *Distributed Consensus*¹

A network consists of N nodes. Each node has an input value (e.g., a block) that they propose to other nodes. Some nodes are faulty (not responding) or malicious, trying to propose a wrong input.

Two properties must hold:

- The process has to terminate with all **honest nodes** in agreement on **the same input value**.
 - The value must have been **generated** by an **honest node**.
-
- In blockchain networks, nodes try to agree on the following information the world state contains:
 - Which of the proposed **transactions** are **confirmed**?
 - In which **order** do the **transactions** appear in the ledger?

Byzantine Generals Problem

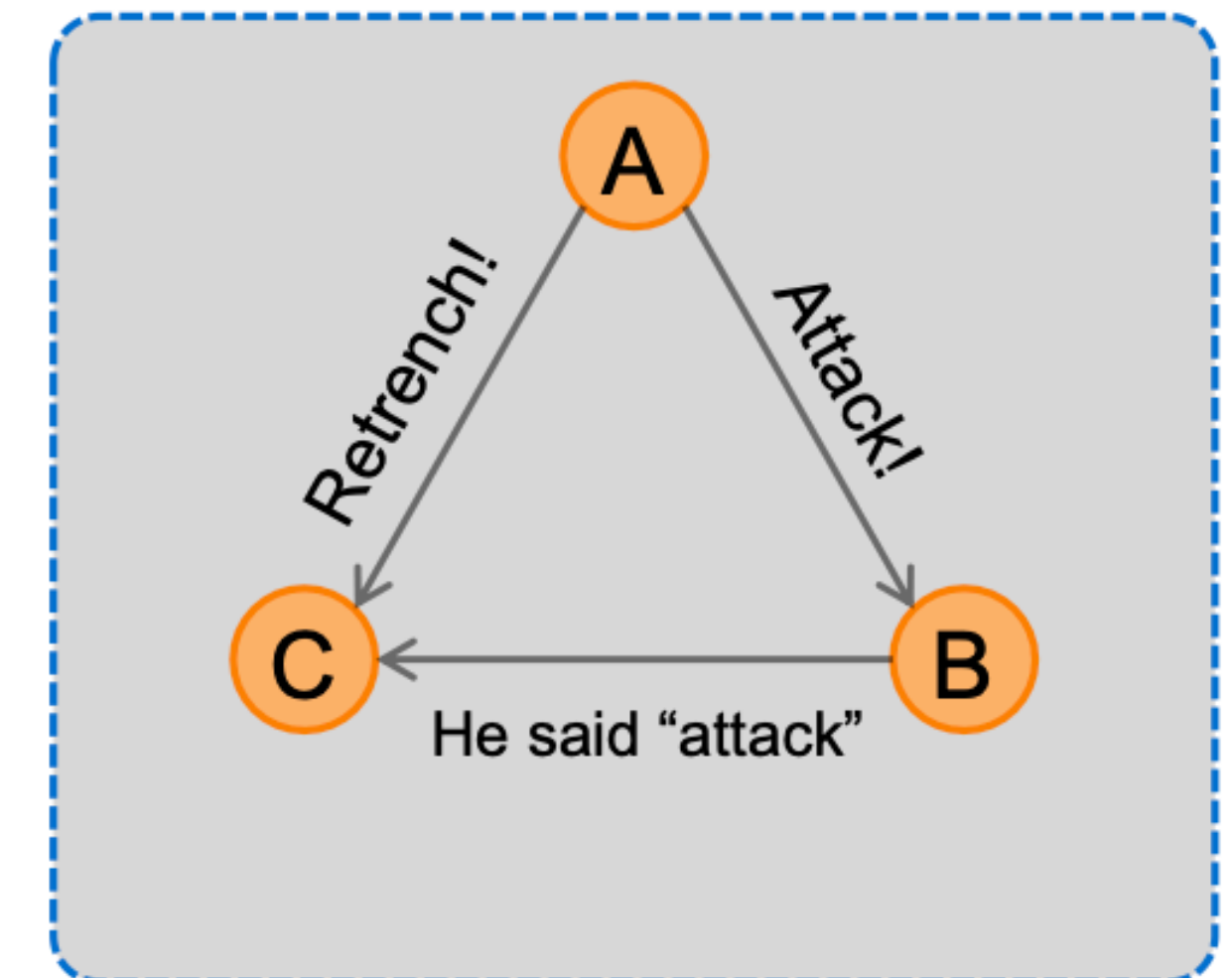
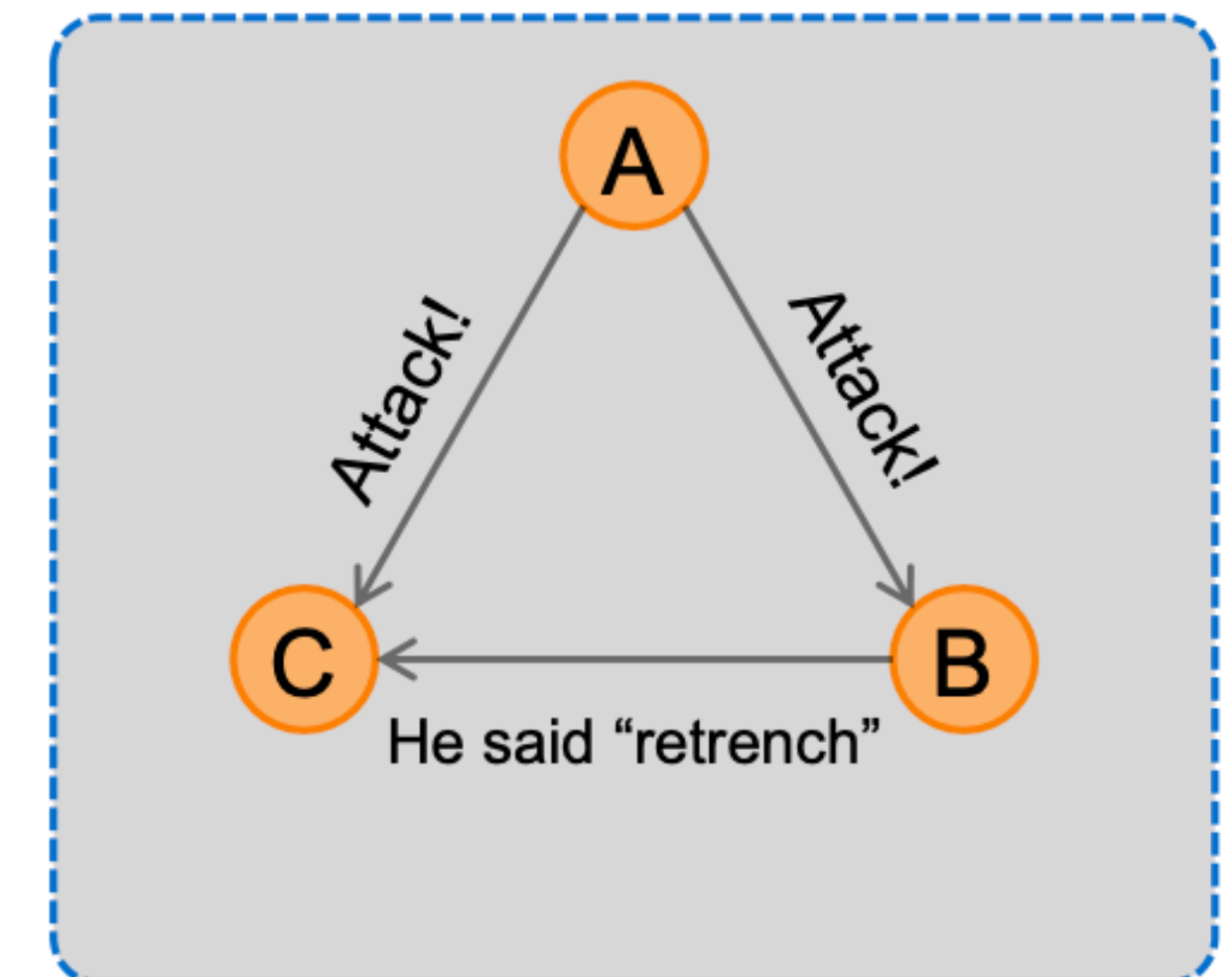
The Byzantine army wants to invade an enemy city. However, it is separated into multiple divisions. They want to attack at the same time, therefore they have to communicate in between the divisions to find a common time to attack.

A general is responsible for one division. These generals communicate by messenger. Some of the generals may be traitors, sending wrong messages to other generals. **The goal is for all loyal generals to derive the same plan without the traitors being able to convince other generals of the wrong plan.**

This property is called “Byzantine-Fault Tolerance” (BFT).

It can be shown that if **more or equal to one third of the generals are malicious**, it is **impossible** for the honest nodes to derive a **common plan** (i.e., consensus cannot be reached).

Three generals

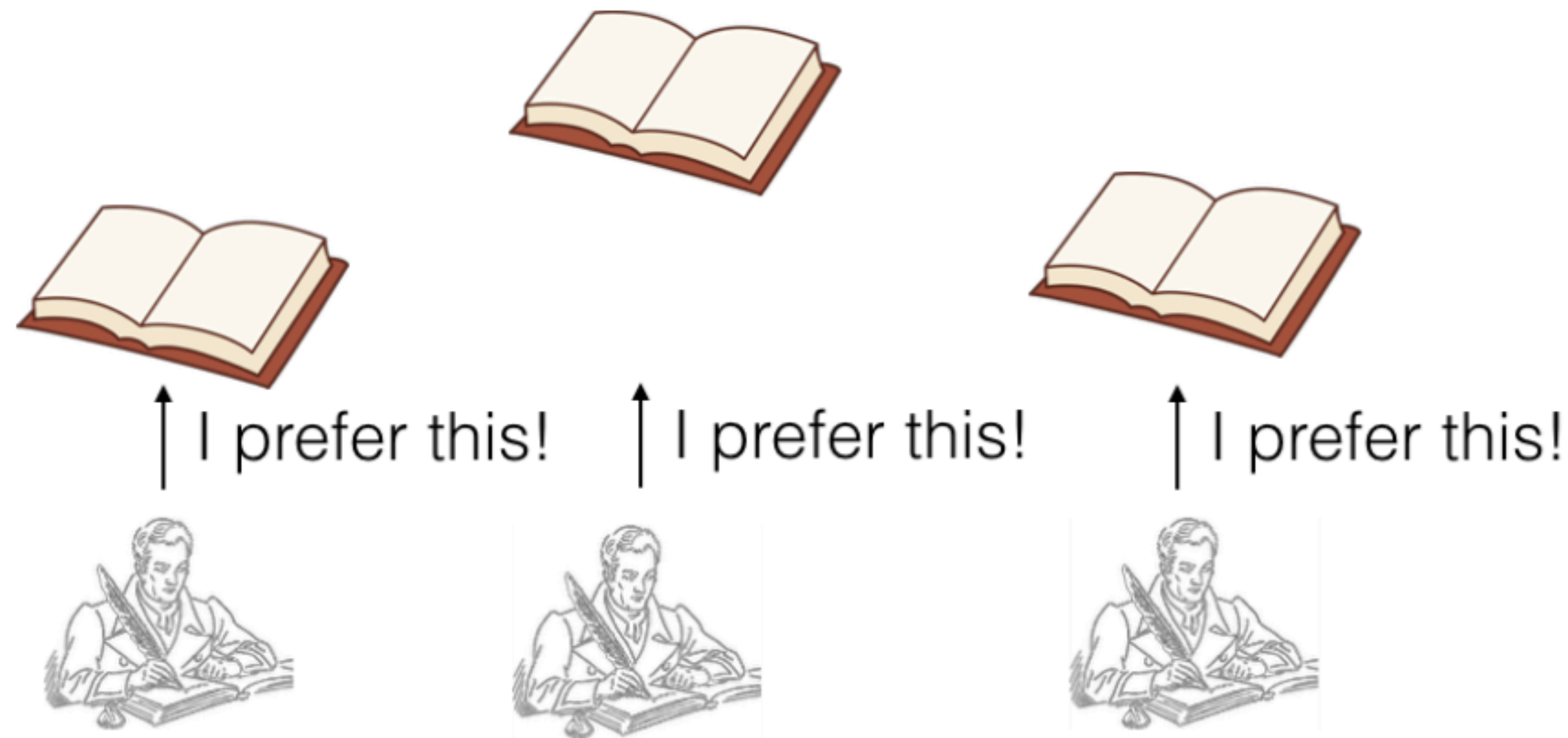


C does not know what to agree on.

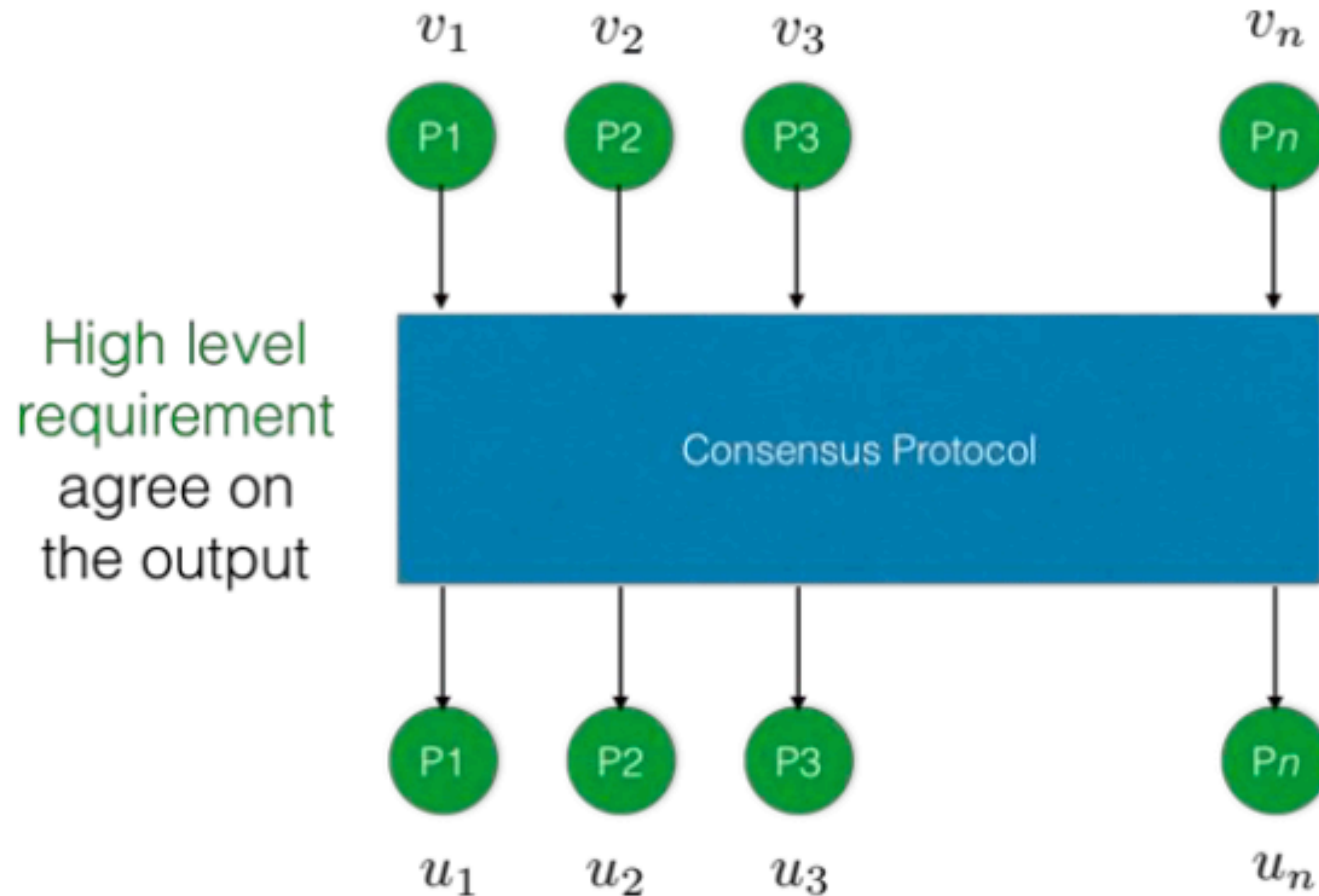
Consensus Models: Motivation

A transaction history and/or state of the service needs to be **agreed** by all servers.

Servers may be operated by participants with **diverging interests**, in terms of the history of transactions and/or state of the service.



Consensus Models: Motivation



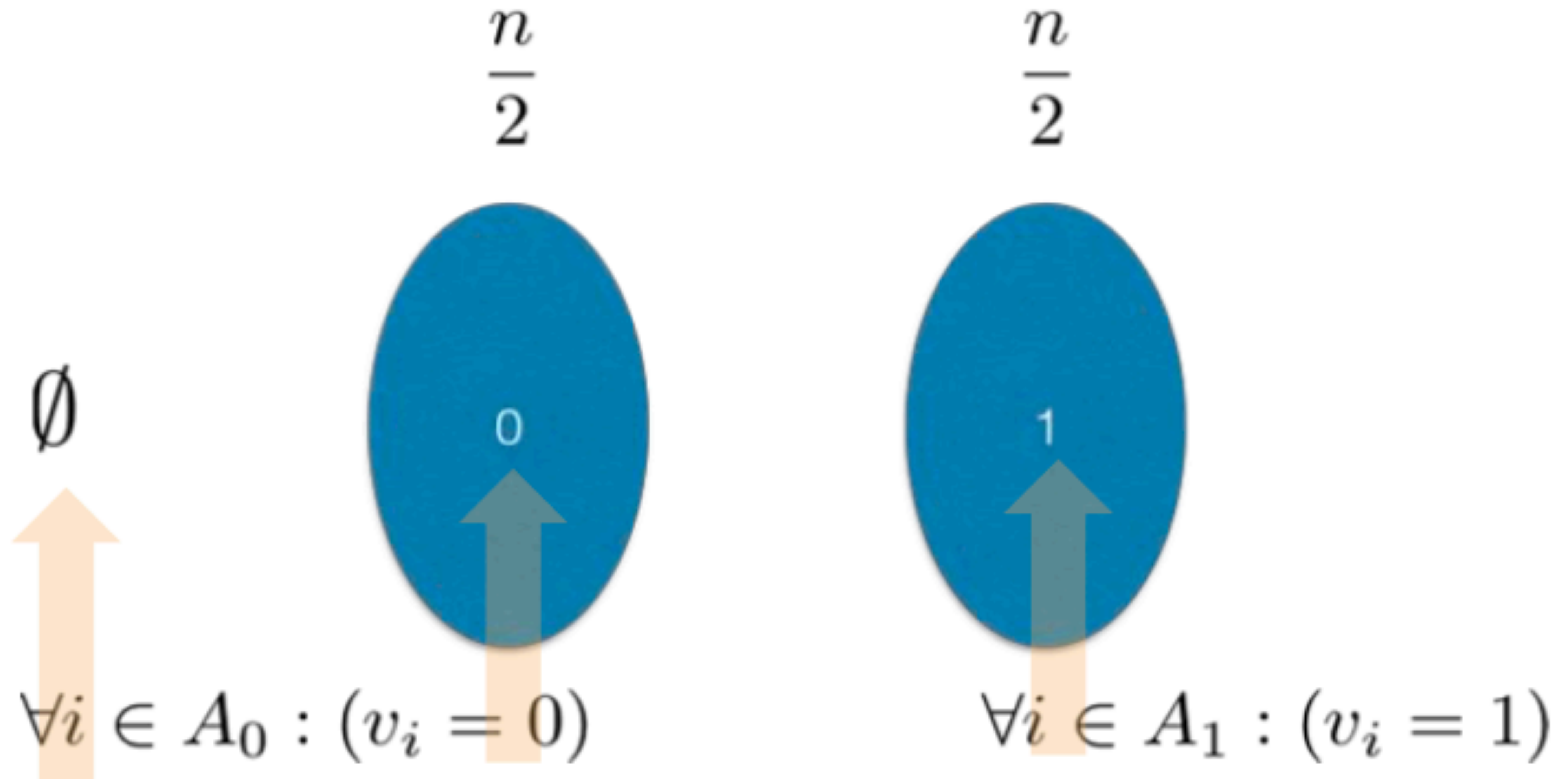
Consensus Models: Properties

Termination $\forall i \in H (u_i \text{ is defined})$

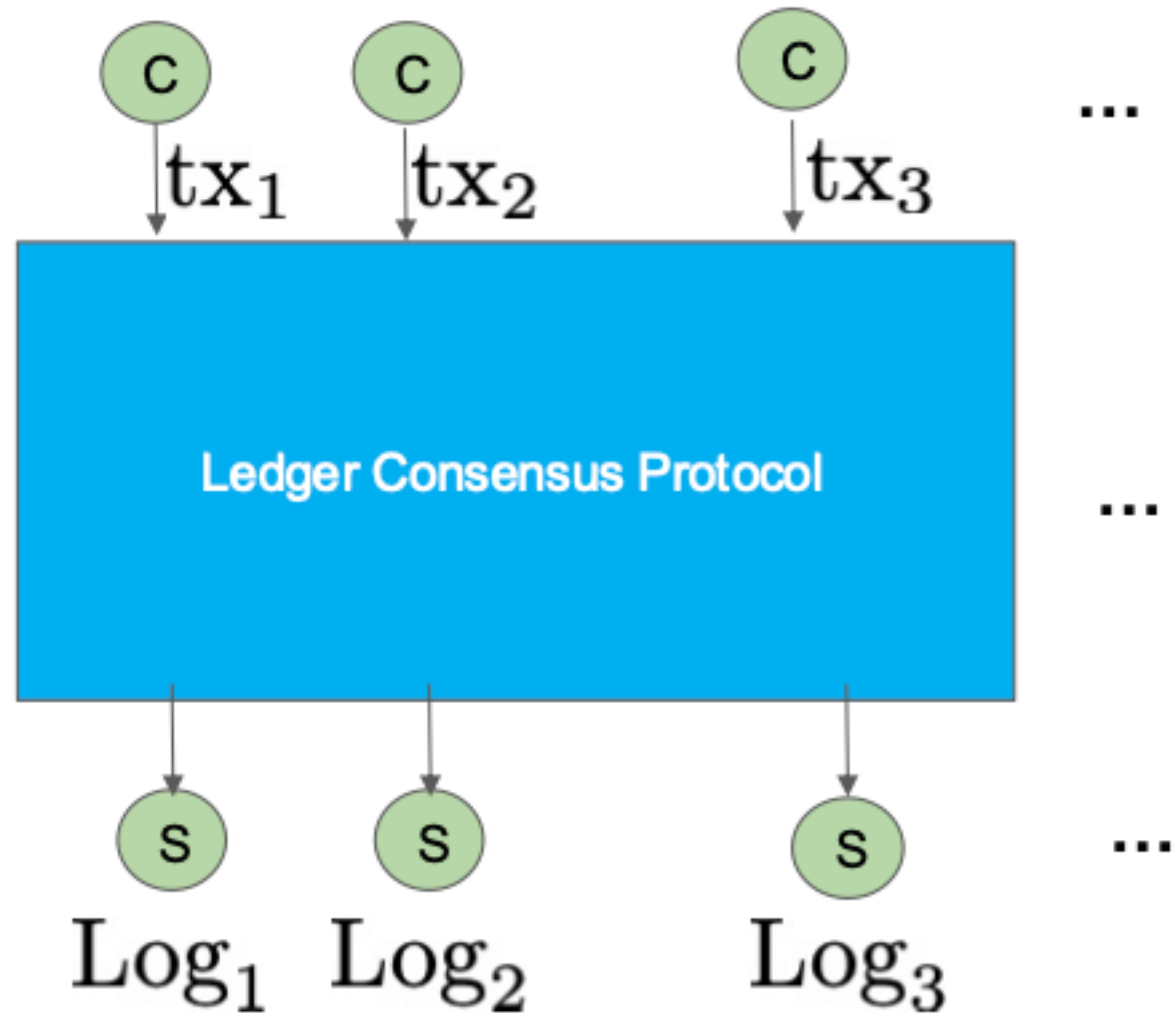
Agreement $\forall i, j \in H (u_i = u_j)$

Validity $\exists v (\forall i \in H (v_i = v)) \implies (\forall i \in H (u_i = v))$

Consensus Models: Honest Majority



Consensus Models: Ledger



Consensus Models: Ledger

Consistency:

$$\forall i, j \in \mathbf{H}, t, t' : (\text{Log}_i[t] \not\preceq \text{Log}_j[t']) \rightarrow (\text{Log}_j[t'] \preceq \text{Log}_i[t])$$

Liveness:

$$(\forall i \in \mathbf{H} : \text{tx} \in I_i[t]) \rightarrow (\forall i \in \mathbf{H} : \text{tx} \in \text{Log}_i[t + u])$$

$I_j[t]$ = Transaction Input of party j at time t consistent with its log

$\text{Log}_j[t]$ = Log of party j at time t

Consensus Models: Ledger

Practical Byzantine Fault Tolerance

Miguel Castro and Barbara Liskov
*Laboratory for Computer Science,
Massachusetts Institute of Technology,
545 Technology Square, Cambridge, MA 02139*
`{castro,liskov}@lcs.mit.edu`

arbitrary behavior. Whereas previous algorithms assumed a synchronous system or were too slow to be used in practice, the algorithm described in this paper is practical: it works in asynchronous environments like the Internet and incorporates

Bitcoin: A Peer-to-Peer Electronic Cash System

Satoshi Nakamoto
`satoshin@gmx.com`
`www.bitcoin.org`

Abstract. A purely peer-to-peer version of electronic cash would allow online payments to be sent directly from one party to another without going through a financial institution. Digital signatures provide part of the solution, but the main benefits are lost if a trusted third party is still required to prevent double-spending. We propose a solution to the double-spending problem using a peer-to-peer network. The network timestamps transactions by hashing them into an ongoing chain of hash-based proof-of-work, forming a record that cannot be changed without redoing

runs uninterruptedly over the Internet since 2009 assuming bounded delays

Consensus Models: Ledger

PBFT

Its operation & threat model features:

- **Asynchronous** communication, point-to-point channels
- Authenticated setting: parties are able to **identify message sources**
- **Fixed number** of participants
- **Computationally bounded** adversary in the sense it cannot forge digital signatures.
- Setup assumption: PKI
- Adversary controls less than $\frac{1}{3}$ of participants (required)
- **No privacy** properties for servers
- **communication complexity** $O(n^2)$ in best case, where n is the number of participants
- No protocol **incentives**

PoW

Its threat model features:

- **Bounded delay** communication by 'diffusion' of messages (no point-to-point channels)
- No ability to **identify message** sources - unauthenticated setting
- **Unknown** and **fluctuating number** of participants ... **dynamic availability** 
- **Computationally bounded** adversary possessing $\sim < 50\%$ of resources
- Setup: **public random string** (no PKI)
- Tolerates potential **spikes** of adversarial majority... **self-heals** 
- **Privacy for** maintainers... protocol transcript maintains anonymity/unlinkability of participants
- **Optimal amortized communication complexity** of single multicast per block.